Sangrah App Overview

Oct 27, 2026 · @Ateeshay

Sangrah (संग्रह, "collection") is an iOS app that saves anything you read, condenses it with Claude, and turns your saves into new content. This document lists everything that exists in the app and backend today, and what is still to come.

What Sangrah does

Sangrah is a personal knowledge library for people who read a lot online and want to use what they read. You share a link, a screenshot or a PDF from any app; Sangrah fetches the content, has Claude write a hook, a summary, key takeaways and tags, and files it in a searchable library. From the library you can combine several saves into one synthesis, draft social posts in your own voice, and export clean PDFs.

Target users are knowledge workers, founders and creators who save articles, threads and newsletters and want insight, not a pile of unread tabs. The app is iOS only (iOS 17+), built in SwiftUI, with a Supabase backend and the Claude API. Pricing is a free tier (50 saves and 10 posts a month, enforced in the UI only today) and a planned Pro tier with unlimited use.

The product name was decided on 27 Oct 2026. Code identifiers still use the old name, Commonplace (bundle id `app.commonplace.ios`, repository `ateeshayjain/commonplace`), until App Store submission.

Screens and flows

The app has three tabs (Library, Search, Profile) plus sheets and pushed screens reached from them.

Screen	What the user can do	State
Sign in	Sign in with Apple; email and password sign-in or account creation; magic-link (OTP) code entry; "Try Demo Mode" with sample data and no backend	Built
Onboarding	Three-page intro with Continue, Skip and Get Started; shown once	Built

Screen	What the user can do	State
Library	Scroll saved items (title, source badge, summary, read time); filter by category chips and by tag chips; sort newest, oldest or favourites; pull to refresh; swipe to favourite, archive or delete; multi-select then "Create Content" to open Compose; "+" to paste a URL (with two sample links)	Built
Document detail	Source and read time; title and author; the AI hook as a pull-quote; AI summary; tags; key points; categories (editable); PDF preview or content excerpt with "Read Full Article"; personal notes; toolbar menu with favourite, share, generate posts, archive, delete	Built
PDF viewer	In-app reading of a saved or generated PDF	Built
Search	Full-text search over title, content and summary; recent searches with Clear; suggested searches; results list with source and date	Built
Compose	Pick 2+ saved items, a content type (post, thread, newsletter, article, summary), target platforms, analysis depth (1st, 2nd or 3rd order) and optional custom instructions; see AI analysis in three tabs plus common themes; copy the generated draft	Built
Campaigns	See connected platforms (X, LinkedIn) and connect more via OAuth; analyse your posting style into a voice profile; generate three post variations per platform with character counts and hashtags; pick a variation; "Generate PDF" export	Built

Screen	What the user can do	State
Profile	Library stats; Weekly Summary (reading insights); Settings for notifications, appearance (theme, font size, line spacing) and categories (add your own); Upgrade to Pro and subscription management placeholders; Contact Support, FAQ, Privacy Policy; Sign Out; version	Built, several rows are placeholders
Share Extension	Save a link from any app's share sheet into Sangrah	Code exists, not yet an Xcode target

The save flow: paste or share a URL, the app creates a document in `pending` state and calls the backend; the backend moves it through fetching, extracting, enriching and rendering to `completed` (or `failed`); the Library updates when you refresh.

What the AI produces

Every AI call goes through one shared module that pins the SDK version, picks the model, and asks for structured JSON so results are never parsed by regex. Claude Opus 5.5 handles per-item enrichment, posts and voice analysis; Claude Fable 5.1 handles multi-document synthesis, with reasoning effort scaled to the chosen depth.

Output	Where it appears	Shape
Hook	Pull-quote at the top of Document detail	One line, at most 150 characters
Short summary	Library row	2 to 3 sentences
Long summary	"AI Summary" box on Document detail	2 to 3 paragraphs, written as bullets
Key takeaways	"Key Points" on Document detail	Exactly 3
Tags	Tag chips on Document detail; tag filter row in Library	Up to 6 per item, reused from your existing tags where they fit, per-user, deduplicated by slug so "Design Trends" and "DesignTrends" merge; Devanagari tags are preserved

Output	Where it appears	Shape
Complexity and reading level	Metadata	1 to 10 score; casual, intermediate, advanced or expert
Synthesis	Compose screen	First-order summary, second-order patterns and connections, third-order implications and contrarian takes, plus common themes and a draft in the chosen format
Social posts	Campaigns screen	Three variations per platform, within 280 characters for X and 3000 for LinkedIn, with hashtags, matched to your voice profile
Voice profile	Campaigns screen, after "Analyze"	Tone, vocabulary level, post structure, hashtag and emoji usage, engagement style, top topics, sample phrases

If the backend has no Anthropic key, enrichment falls back to a plain excerpt of the content so saving still works.

Supported sources

The app recognises the source from the URL and shows a matching badge. Extraction uses a reader service that returns the page text, so anything with readable HTML works today.

Source	Today	Planned
Web articles, news, Substack, Medium, Hacker News	Saved and enriched	Keep images and Markdown structure
X / Twitter posts and threads	Saved when the page text is reachable; detected as `twitter`	Thread stitching
LinkedIn posts and articles	Detected; text extraction where public	
Reddit	Detected; page text extraction	

Source	Today	Planned
PDFs	Saved and viewable in app	Text extraction for enrichment
Screenshots and WhatsApp text	Source types exist (`screenshot`, `manual`)	Capture flow and OCR
YouTube	Detected; only page text today	Transcript and chapters
Instagram, Threads, podcasts	Detected only	Captions, carousel images, audio transcripts
Book pages (OCR)	Data model only	Camera capture and on-device OCR
Email	Source type exists	Forwarding address

The agreed order is to ship the low-friction sources first (articles, X, Substack, Reddit, pasted text, screenshots, links, PDFs) and leave YouTube and Instagram, which block scraping, for later.

Backend

The backend is a live Supabase project (`commonplace`, Mumbai region) with Postgres, Auth, Storage and Edge Functions. Every table has row-level security so a user can only read and write their own rows.

Tables

Table	Holds
`users`	Profile, subscription tier, preferences
`documents`	Every saved item: URL, source type, title, author, content, status, AI fields (hook, summaries, key points, complexity, reading level), categories, favourite and archive flags, read progress, notes, metadata JSON
`categories`	System categories (Technology, Business, Health, Science, Culture, Politics, Design, Personal Dev) and user-made ones
`tags`, `document_tags`	Per-user tags with a generated slug, linked to documents

Table	Holds
`collections`, `collection_documents`	Named groups of saves, optionally public (shows the list, not the contents)
`generated_assets`	Carousels, quote graphics, video scripts and dossiers, with status and storage paths (schema ready, no generators yet)
`social_connections`	OAuth tokens per platform
`voice_profiles`, `campaigns`, `campaign_posts`	Style analysis and generated posts

Edge Functions (Deno, all deployed)

1. `process-url`: creates the document, then in the background fetches the page, enriches it with Claude, writes the hook, summaries, key points and tags.
2. `synthesize-content`: multi-document analysis at the chosen depth.
3. `generate-posts`: three variations per platform, voice-matched.
4. `analyze-style`: builds a voice profile from the user's past posts.
5. `oauth-connect`: X and LinkedIn OAuth with PKCE.
6. `generate-pdf`: clean PDF of a document, uploaded to storage.

Storage: a public `pdfs` bucket for generated PDFs and a private `generated-assets` bucket scoped per user.

Auth: Sign in with Apple, email and password, and magic-link codes, all through Supabase Auth. Tokens live in the iOS Keychain.

CI: every push builds the iOS app on a macOS runner and typechecks and tests the Edge Functions, so changes can be verified without pulling the code locally.

Build status

As of 27 Oct 2026 the backend is deployed and the iOS app compiles on CI and in Xcode. The app has not yet been run end to end against the live backend with a real Anthropic key.

PRD module	Feature	Status
iOS	Library, Search, Detail, Profile, Auth, Onboarding	Live
iOS	Share Extension	Code written, not in an Xcode target

PRD module	Feature	Status
iOS	Camera and document scanning	Not started
iOS	Offline cache	Not started
1 Ingestion	Web, Substack, news, Reddit, X page text	Live (plain text)
1 Ingestion	YouTube transcripts, Instagram, Reel audio	Not started
1 Ingestion	Book OCR	Schema only
2 AI	Hook, summaries, key takeaways, auto-tags	Live
2 AI	Multi-document synthesis	Live
2 AI	Social posts and voice profile	Live
3 PDF	Single-document PDF	Live
3 PDF	Multi-document dossier with table of contents	Not started
4 Generation	Carousel, quote graphic, video script	Types defined, no generator
Platform	Collections UI	Tables exist, no screens
Platform	Subscription tiers and payments	UI limits only, no StoreKit, no server enforcement
Platform	Branding (icon, accent colour)	Name decided (Sangrah); icon and accent open

Open items

Before the first real end-to-end test:

- [ ] Add the Anthropic API key in the Supabase dashboard (Edge Functions, Secrets). Until then saves complete without AI output.
- [ ] Run the app on a phone from Xcode, sign in, save one URL, and confirm hook, summary and tags appear.
- [ ] Add the Share Extension as an Xcode target so links can be saved from Safari and other apps.

Before launch:

- [ ] App icon and accent colour, per the branding guidelines in the repository (`docs/branding.md`).
- [ ] Move bundle id, App Group and URL scheme from `commonplace` to `sangrah` and set up signing for them.
- [ ] Screenshot and pasted-text capture; PDF text extraction.
- [ ] Collections screens and the multi-document PDF dossier.
- [ ] Subscription with StoreKit 2 and server-side limits.
- [ ] App Store privacy labels, screenshots and review notes.

The detailed phase plan, dependencies and competitive research live in the repository in `project.md` and `docs/competitive-landscape.md`.